

# PETSc Rewrite of Slope Stability

## From the legacy MATLAB workflow to the current PETSc runtime

- Translate the original MATLAB mental model into the current executable PETSc/Python path.
- Show where parallel ownership, PMG, unified runners, unified meshes, and unified visualisation changed the design.

Repository-local material only. Snapshot date: April 13, 2026.

1. MATLAB baseline
2. What changed structurally
3. What is new and reusable
4. How to add a new benchmark
5. Unified visualisation
6. Unified meshes
7. Speed comparison
8. Where to edit the code and close

## Legacy MATLAB Entry Points

3D\_hetero\_SSR.m  
+CONTINUATION/\*  
+NEWTON/\*  
+CONSTITUTIVE\_PROBLEM/\*  
+MESH/\*  
+VIZ/\*

## PETSc Runtime Entry Points

README.md  
run\_config.py  
run\_case\_from\_config.py  
execution/asset\_case/runner.py

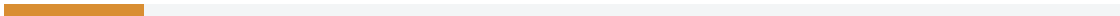
## Local Docs And Studies

petsc\_matlab\_map  
config-scheme-3d.md  
config-case-matrix.md  
performance\_comparison

## How the material is used

Architecture statements in this presentation are anchored on the default 3D heterogeneous SSR P4/PMG benchmark, because it exercises the current mainline path. Runtime comparisons later in the presentation come from the committed P2 study, so architecture and performance evidence stay separate.

# MATLAB Baseline



# The Legacy Execution Model Is Script-Centric

- One benchmark usually equals one top-level script.
- `slope_stability_3D_hetero_SSR.m` declares the case, chooses the solver, launches continuation, and plots the results.
- The numerical kernels are modular, but orchestration remains case-local rather than config-driven.
- The workflow is learned by reading the script from top to bottom.

```
start
→ quadrature
→ local basis
→ mesh load
→ material expansion + labels
→ K_elast, B, f_V
→ constitutive object
→ SSR continuation
→ MATLAB figures
```

| Package               | Role                     | Typical content                                                               |
|-----------------------|--------------------------|-------------------------------------------------------------------------------|
| +ASSEMBLY             | FE preprocessing         | Quadrature, local basis, elastic stiffness, volume forces, material expansion |
| +CONTINUATION         | Outer branch tracking    | Direct SSR, indirect SSR, LL, init phase, omega stepping                      |
| +NEWTON               | Nonlinear solve core     | Damped Newton, indirect SSR Newton, LL Newton                                 |
| +CONSTITUTIVE_PROBLEM | Stress and tangent model | Davis reduction, stress evaluation, tangent assembly, potential               |
| +LINEAR_SOLVERS       | Linear solver factory    | Direct, AGMG, ICHOL, HYPRE BoomerAMG, DFGMRES, collectors                     |
| +MESH, +VIZ, +SEEPAGE | Domain-specific helpers  | Case loaders, plotting, seepage preprocessing and flow solve                  |

## Common benchmark solve flow

- The benchmark script selects the analysis, continuation path, solver handle, and stopping settings.
- The nonlinear path then branches into direct SSR, indirect SSR, or indirect LL continuation.
- Each continuation path calls a matching Newton kernel and damping path.
- Accepted steps extend the history arrays; rejected steps shrink and retry.

## Main nonlinear branches

|                     |                                                        |
|---------------------|--------------------------------------------------------|
| <b>Direct SSR</b>   | SSR_direct_continuation.m →<br>NEWTON.newton           |
| <b>Indirect SSR</b> | SSR_indirect_continuation.m<br>→ NEWTON.newton_ind_SSR |
| <b>Indirect LL</b>  | LL_indirect_continuation.m<br>→ NEWTON.newton_ind_LL   |

## Constitutive kernel role

- CONSTITUTIVE centralises Davis reduction, stress evaluation, tangent evaluation, residual assembly, and timing collectors.
- The object owns the integration-point material fields and mediates calls into the lower-level constitutive functions.
- This is already the closest MATLAB analogue of the PETSc constitutive operator.

script / continuation  
→ constitutive object  
→ reduction + stress  
→ tangent + residual  
→ Newton / continuation history

The constitutive wrapper already exists in MATLAB. The rewrite changes the ownership and assembly model around it.



## Mesh, Materials, And Dirichlet Labels In MATLAB Are Closely Coupled

| MATLAB path                       | What it assumes                              | Consequence                                                    |
|-----------------------------------|----------------------------------------------|----------------------------------------------------------------|
| load_mesh_P2.m                    | Quadratic tetrahedral HDF5 input             | Benchmark meshes and FE degree are tied together               |
| load_mesh_gmsh_waterlevels.m      | Case-specific Gmsh seepage conventions       | Loader logic lives with the case family                        |
| midpoints_P2.m,<br>midpoints_P4.m | Degree-specific node generation              | Higher order support grows through dedicated utilities         |
| Q mask from face labels           | BC labels are interpreted in the loader path | Free-DOF extraction depends on the loaded mesh convention      |
| material_identif expansion        | Material ids are resolved during loading     | Material heterogeneity is coupled to the same loader decisions |

This is one of the biggest conceptual changes in the rewrite: the PETSc side moves toward **canonical mesh family storage** plus **solver-side degree selection**, instead of one file format per numerical path.

## What the MATLAB flow still does well

- The benchmark script can call `VIZ.*` immediately after the solve and inspect the same assembled state.
- That keeps the familiar 3D and slice figures close to the benchmark logic the original authors already know.

## What stays coupled

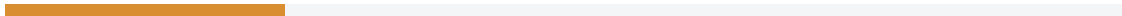
|              |                                                           |
|--------------|-----------------------------------------------------------|
| <b>state</b> | live MATLAB workspace after the solve                     |
| <b>plots</b> | benchmark-local <code>VIZ.*</code> calls and figure setup |
| <b>reuse</b> | copy-and-adapt workflow between benchmark families        |

start

- solve benchmark
- call `VIZ.*`
- inspect figure state
- export figure manually
- repeat per case family

The figure family stays comparable. The production path changes: case-local plotting code and live interpreter state are replaced by shared exports and notebook reconstruction.

# What Changed Structurally



# What Stays Familiar, What Changes

## Familiar problem structure

- SSR and LL as the benchmark families people already recognize
- Newton plus step accept-or-shrink logic
- Davis reduction and the constitutive kernel as the physical core
- continuation outputs such as  $\lambda$ ,  $\omega$ , and  $u_{\max}$

## Changed runtime structure

- top-level script orchestration
- one global sparse rebuild per Newton step
- one solver-handle-only view of the linear solve
- workspace-local outputs and plotting state

The mathematics is still familiar. The execution packaging, ownership model, and reusable interfaces are what changed.

## Responsibility Map: MATLAB Driver To PETSc Modules

| Role                             | MATLAB location              | PETSc location                  |
|----------------------------------|------------------------------|---------------------------------|
| Case declaration                 | Top of the benchmark script  | case.toml + run_config.py       |
| Runner dispatch                  | Implicit in the script name  | run_case_from_config.py         |
| Mesh and material preprocessing  | Script body + MESH/+ASSEMBLY | mesh/* + fem/* + runner setup   |
| Constitutive handle construction | CONSTITUTIVE.m               | problem.py:ConstitutiveOperator |
| Continuation launch              | SSR_indirect_continuation.m  | continuation/indirect.py        |
| Linear solver factory            | set_linear_solver.m          | solver.py:SolverFactory         |

## Walk-through benchmark

- The checked-in 3D heterogeneous SSR default benchmark is the architecture anchor.
- It keeps the owned-row, overlap, pmg\_shell, and P4 path in one place.

```
analysis = ssr                                elem_type = P4
node_ordering = block_metis                   mpi_distribute_by_nodes = true
constitutive_mode = overlap                   tangent_kernel = rows
pc_backend = pmg_shell
```

```
start
→ load config → resolve asset and route → build and reorder mesh
→ build owned elastic rows → prepare owned tangent pattern
→ create constitutive operator → build PETSc solver
→ launch indirect SSR continuation
```

### Config side

- `core/run_config.py` loads TOML, validates `elem_type`, analysis kind, and the asset-first public section shapes.
- The config object preserves the same high-level blocks MATLAB already used: problem, continuation, Newton, linear solver, and outputs.
- Materials, hydraulics, and boundary conditions are resolved from `meshes/<asset>/definition.py`, not local config rows.

```
cfg = load_run_case_config(...)
asset = resolve_problem_asset(cfg)
route = select_case_route(cfg)
result = run_case_config(cfg)
write debug bundle
write history
write VTU
```

## Policy inputs

- `solver_type`
- `mpi_distribute_by_nodes`
- `constitutive_mode`
- `node_ordering`

## Default outcome

- owned tangent path enabled
- lightweight MPI elastic path enabled
- node numbering aligned to ownership through `blockmetis`

## Why this matters

MATLAB kept one global sparse operator in one address space.

The PETSc rewrite makes row ownership explicit early, because that ownership controls:

- local element overlap
- constitutive work placement
- matrix assembly
- preconditioner construction



### Prepared once

- rank-owned node interval
- overlap elements for those rows
- local CSR matrix pattern
- elastic values on that pattern
- constrained diagonal positions

### Refreshed every Newton step

- constitutive state on overlap integration points
- internal force rows
- tangent values on the fixed CSR pattern
- regularised matrix blend  $K_r$

This is the deepest structural change relative to MATLAB: the rewrite no longer treats every Newton step as a fresh global sparse rebuild. The **structure** is prepared once; the **values** are refreshed in place.

## Overlap constitutive mode

- Each rank evaluates the constitutive model on the overlap integration points needed by its owned rows.
- The default path avoids a unique-owner recovery stage.
- The tangent kernel receives local overlap data directly.

## Rows kernel

- The kernel walks the fixed owned CSR pattern row by row.
- Metadata such as `row_slot_ptr`, `slot_elem`, `slot_lrow`, and `slot_pos` replace the old “assemble element matrix, then scatter” view.
- The bilinear form is unchanged; the dataflow is not.

- The fine-grid operator is the reduced free-space regularized operator coming from the owned-row tangent path.
- The outer Krylov family is still the PETSc-wrapped MATLAB-style deflated FGMRES interface.
- The actual preconditioner is a custom shell V-cycle, not the built-in PETSc PCMG path.
- The hierarchy metadata enters through `pmg_hierarchy` and stays aligned with the same reordered free-space numbering as the solve operator.

---

**MATLAB intuition**

one current sparse system plus one solver handle

---

**PETSc mainline**

current reduced operator plus shell hierarchy plus chosen outer KSP

---

In this repository, “parallel PMG” refers to this shell-preconditioned reduced solve path.

# How The PMG Hierarchy Is Built And Rebuilt

runner startup:

build reordered pmg\_hierarchy

default P4 chain:  $P1 \rightarrow P2 \rightarrow P4$

store free-space levels + prolongations

Newton step  $n$ :

pass current reduced operator

validate level sizes + ownership

configure shell V-cycle on live matrix

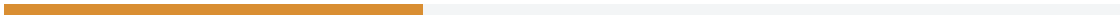
- The hierarchy geometry and transfer metadata are built once from the reordered mesh family, element order, and material rows.
- `_ensure_pmg_state()` then turns those level-to-level transfers into PETSc matrices.
- The rebuild policy is still `every_newton`: the shell preconditioner is reconfigured on the current operator each Newton step.
- So the multilevel layout is stable, while the fine-grid matrix and smoother state stay current.

## Current PMG Shell Versus MATLAB HYPRE Intuition

| Question             | MATLAB HYPRE when selected                        | Current P4 mainline                                                                      |
|----------------------|---------------------------------------------------|------------------------------------------------------------------------------------------|
| Outer Krylov wrapper | script-selected DFGM-RES + HYPRE                  | PETSc-wrapped deflated FGMRES from <code>solver_type</code>                              |
| Preconditioner core  | BoomerAMG through the MATLAB binding              | <code>pmg_shell</code> V-cycle on the reduced operator                                   |
| Coarse solve         | internal to BoomerAMG                             | <code>preonly</code> + <code>hypre(boomeramg)</code> on the coarse PMG level             |
| Level smoother       | not exposed as a shell hierarchy                  | <code>chebyshev+jacobi</code> on robust parallel shell; else <code>richardson+sor</code> |
| Rebuild timing       | <code>solver</code> handle reused from the script | shell configured on the current tangent each Newton step                                 |

MATLAB HYPRE is still the right coarse-solve intuition. The current mainline simply wraps it in a visible shell PMG hierarchy and an explicit outer Krylov surface.

# What Is New And Reusable



## What Became Public And Reproducible

| Surface              | Examples                                                                                 | Why it is useful                                                 |
|----------------------|------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| [continuation]       | predictor, omega controller, $\Delta\lambda$ limits, warm-start and fine-switch controls | continuation behavior stops being script-local                   |
| [newton]             | stop criterion, stop tolerance, damping and line-search mode                             | Newton policy becomes visible and comparable                     |
| [linear_solver]      | outer solver, PC backend, rebuild policy, HYPRE/GAMG/BDDC options                        | solver studies move into config instead of benchmark-local edits |
| [execution]          | node ordering, ownership mode, tangent kernel                                            | parallel structure is now a public decision                      |
| [export] + notebooks | standard output names, notebook metadata, slice planes                                   | postprocessing stops depending on live workspace state           |

```
start
→ run case from config
→ save run_debug.h5
→ save continuation_history.json
→ save final_solution.vtu
→ save resolved_config.toml
→ reuse them in generated notebooks and external tools
```

## What this improves

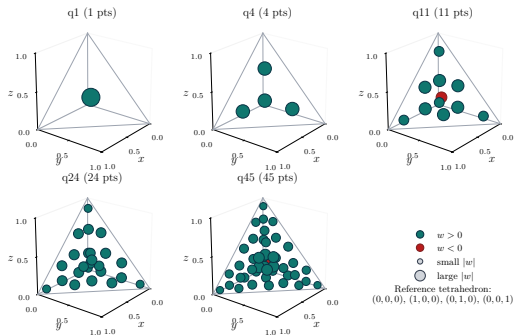
- reproducibility
- postprocessing outside the solver process
- notebook regeneration without rerunning the case
- standard field names for PyVista and ParaView

## MATLAB comparison

- MATLAB often kept these objects in workspace structs or figure state.
- The rewrite promotes them into stable file outputs.
- That is what enables the “unified visualisation” story later in the deck.



# P4, Quadrature, And 3D Export Are Real Runtime Paths



- The config and element helpers accept P1, P2, and P4 on the public interface.
- The VTK export path supports VTK\_LAGRANGE\_TETRAHEDRON for P4, including correct node ordering.
- The postprocess layer adds pointwise deviatoric strain export for 3D P4, so the higher-order benchmark path can be visualised without dropping back to a low-order fallback.
- This matters in practice: higher-order runs, quadrature choice, and the final visual product are now on the public runtime path instead of being one-off utilities.

# How To Add A New Benchmark



| Task                     | MATLAB habit                                        | PETSc rewrite habit                                                                                             |
|--------------------------|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Select the runner family | pick a top-level script                             | copy a benchmark folder and set <code>problem.asset</code> plus <code>problem.analysis</code>                   |
| Select the physics path  | change SSR / LL / seepage calls in the script       | set <code>problem.analysis</code> ; use <code>continuation.method</code> only where the runner exposes it       |
| Point at the mesh input  | edit the loader call inside the script              | select <code>problem.mesh_variant</code> and optional <code>problem.profile</code>                              |
| Define heterogeneity     | edit material and conductivity arrays in the script | define materials, mechanics BCs, hydraulics, and seepage BCs in <code>meshes/&lt;asset&gt;/definition.py</code> |
| Run and inspect outputs  | execute the script and keep workspace state alive   | run <code>case.toml</code> , then reuse <code>artifacts/simulation/exports/*</code>                             |

# Benchmark Folder Contract And First Run

## Bootstrap once

```
$ ./bootstrap.sh  
$ BOOTSTRAP_MODE=wheel ./bootstrap.sh
```

## Folder contract and first run

```
benchmark_name/  
case.toml  
run.sh  
README.md  
simulation.ipynb  
visualisation.ipynb  
  
$ ./run.sh  
$ ../.venv/bin/python -m  
slope_stability.cli.run_case_from_config  
benchmarks/<case>/case.toml
```

- `case.toml` is the only document the runner consumes directly.
- `run.sh` is a wrapper around the same config-driven entry point.
- `README.md` and the notebooks describe or reuse that config; they do not define solver behavior.
- Standard outputs land under `artifacts/simulation/exports/`; [notebook] only tunes notebook and display defaults.

## Choose The Runner Family First

| Decision                                                            | Typical values                         | What it actually selects                                                                                              |
|---------------------------------------------------------------------|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| problem.asset                                                       | 3d_hetero_slope,<br>3d_hetero_seepage  | the registered problem asset and its definition file                                                                  |
| problem.mesh_variant                                                | asset-local .msh variant               | the concrete canonical Gmsh mesh for that asset                                                                       |
| problem.profile                                                     | fixed_base,<br>roller_base, or omitted | named mechanics BC overrides from the asset definition                                                                |
| problem.analysis<br>[continuation],<br>[newton],<br>[linear_solver] | ssr, ll, seepage<br>numerical controls | the mechanics, limit-load, or seepage-only route<br>solver policy only; not geometry, materials, or<br>hydraulic data |

## What The Current Public Surface Actually Supports

| Route style                   | Analysis | Current continuation surface                                             | Problem input                                  |
|-------------------------------|----------|--------------------------------------------------------------------------|------------------------------------------------|
| 2D mechanics assets           | ssr, ll  | continuation route selected by config                                    | asset, mesh_variant, optional profile          |
| 3D mechanics assets           | ssr, ll  | indirect in the current config-driven mainline                           | asset, mesh_variant, optional profile          |
| Seepage-only assets           | seepage  | no continuation branch; seepage and linear-solver blocks drive the solve | seepage spec from definition.py                |
| 3D seepage-coupled SSR assets | ssr      | indirect SSR after seepage solve                                         | mechanics and seepage specs from definition.py |

case.toml selects assets and numerical controls. Problem-specific physics belongs in meshes/<asset>/definition.py.

## Dry Mechanical Example: 3D Heterogeneous SSR

```
[problem]
asset = "3d_hetero_slope"
mesh_variant = "adaptive_family_a_l1.msh"
analysis = "ssr"
elem_type = "P4"

[execution]
node_ordering = "block_metis"

[continuation]
method = "indirect"
omega_max = 6.7e6

[newton]
stopping_criterion = "absolute_delta_lambda"
stopping_tol = 1e-4

[linear_solver]
pc_backend = "pmg_shell"
```

- This selects the 3D dry SSR runner and the current distributed P4/PMG path.
- `continuation.method = "indirect"` matches the current config-driven 3D mainline.
- The mesh variant resolves through `meshes/3d_hetero_slope/definition.py`; BC labels and materials are asset-owned.
- `[execution]`, `[continuation]`, `[newton]`, and `[linear_solver]` own the parallel and solver policy; `[notebook]` only affects notebook defaults.

## Seepage-only

```
[problem]
asset = "3d_hetero_seepage"
mesh_variant = "concave_family_b.msh"
analysis = "seepage"
[seepage]
linear_tolerance = 1e-10
```

## Seepage-coupled SSR

```
[problem]
asset = "3d_hetero_seepage_transition"
mesh_variant = "transition_default.msh"
profile = "fixed_base"
analysis = "ssr"
[continuation]
method = "indirect"
```

[seepage] carries numerical seepage solver tolerances only. Conductivity, water unit weight, and head conditions live in the selected asset definition.



```
ASSET = build_problem_asset_3d(  
    asset_id="3d_hetero_slope",  
    mesh_variants={...},  
    materials={...},  
    region_assignment={...},  
    mechanics={... profiles ...},  
    seepage=build_seepage_spec(...)  
)
```

- Canonical Gmsh physical names use region:\*, boundary:\*, nodeset:\*, and optional boundary\_geom:\*.
- The asset API maps regions to material ids and builds mechanics and seepage masks from named targets.
- Named profiles replace integer boundary switches such as glued-base or roller-base modes.
- case.toml should not contain raw mesh paths, local material rows, conductivity, water unit weight, or head BCs.
- Generic mesh IO reads geometry and labels; problem ownership stays under meshes/<asset>/definition.py.

## Benchmark Authoring Checklist

1. Copy the nearest benchmark folder under `benchmarks/`, rename it intentionally, and update `[benchmark]`.
2. Set `[problem]` first: `asset`, `mesh_variant`, optional `profile`, `analysis`, and `elem_type`.
3. Put geometry, materials, mechanics BCs, hydraulics, seepage BCs, and profiles in `meshes/<asset>/definition.py`.
4. Fill `[execution]`, `[continuation]`, `[newton]`, and `[linear_solver]`; add `[seepage]` only for numerical seepage controls.
5. Run `./run.sh` or `run_case_from_config`, then verify `artifacts/simulation/exports/*`.
6. Keep `README.md`, `simulation.ipynb`, and `visualisation.ipynb` aligned with the same `case.toml`.

A benchmark should remain explainable from `case.toml`, `meshes/<asset>/definition.py`, and the output folder. If that is no longer true, too much logic has drifted back into hidden runner code.

# Unified Visualisation



start

- solve case
- write final\_solution.vtu + debug bundle + history JSON
- rebuild case mesh from config
- build standard field exports
- generated notebooks / PyVista / ParaView consume the same files

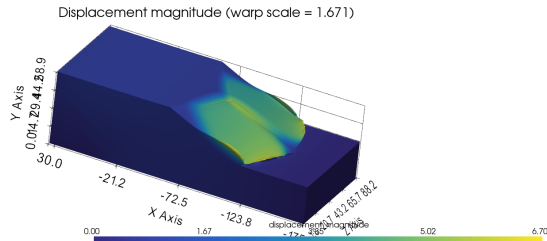
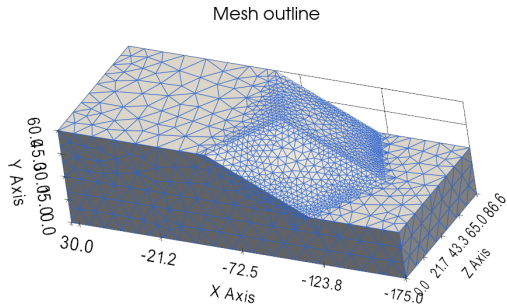
## Key rewrite helpers

- postprocess/case\_mesh.py
- postprocess/field\_exports.py
- benchmarks/notebook\_support.py

## Result

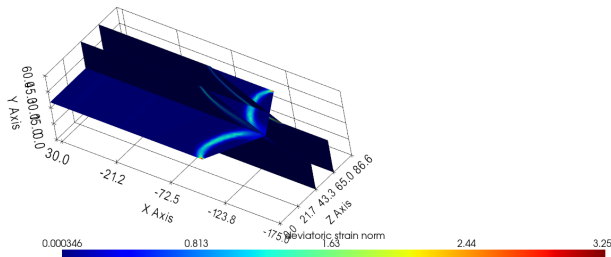
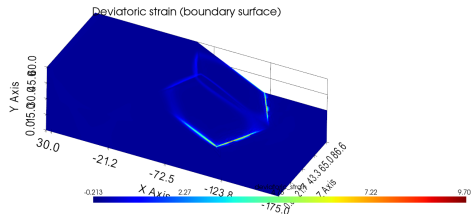
- one mesh reconstruction surface
- one field naming surface
- one notebook generation surface
- one export surface for external viewers

# PETSc 3D Views: Geometry And Warped Displacement



- Both panels are taken from the default indirect 3D SSR visualisation notebook path.
- Left: mesh-outline view showing the local refinement pattern.
- Right: warped displacement view from the same benchmark and the same exported solution bundle.

# PETSc Localisation Surface And Top-View Slices



- Both panels come from the same default indirect visualisation path and the same exported `deviatoric_strain` field.
- The 3D panel keeps the notebook-style full boundary-surface view that the authors are already used to reading.
- The slice panel stays a real top-view cut product at the benchmark-configured planes, with the same capped warm-to-cool scale used in the 3D deviatoric view shown on the left.

# MATLAB And PETSc Reach Similar Slice Products Through Different Workflows

## MATLAB

MATLAB: X = -71.01



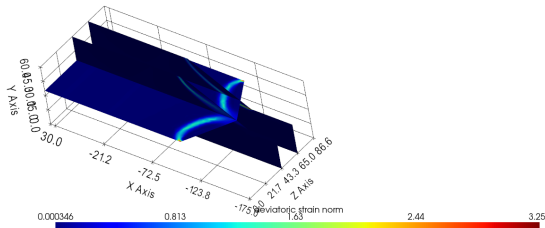
MATLAB: Y = 30.08



MATLAB: Z = 44.71



## PETSc rewrite



The visual end product stays familiar. The production path changes: MATLAB builds the slice product in the case script, while the rewrite rebuilds it from shared exports and shared notebook support.

# Unified Meshes





## MATLAB Mesh Handling And Element Degree Are Entangled

| Question                       | MATLAB answer                                         | Consequence                                          |
|--------------------------------|-------------------------------------------------------|------------------------------------------------------|
| Mesh storage choice            | Usually a degree-specific loader                      | Mesh storage and FE order drift together             |
| Higher-order support path      | Build new midpoint utilities and matching assumptions | Higher order grows sideways by utility proliferation |
| Material and BC interpretation | Inside the case loader path                           | Harder to reuse across benchmark families            |
| Free-mask source               | Loader-specific face conventions                      | Less separation between geometry and solve policy    |

```
ASSET = build_problem_asset_3d(  
  asset_id="3d_hetero_slope",  
  default_variant="adaptive_family_a_l1.msh",  
  mesh_variants={...},  
  mechanics={profiles, materials, BCs}  
) }
```

- Each asset declares mesh variants, profiles, boundary labels, materials, and hydraulics.
- The case config selects the asset, mesh variant, optional profile, analysis, and solver settings.
- The runtime receives reusable asset data, not problem-specific source modules.

## Mesh Family And `elem_type` Are Now Separate Concepts

| Concept                        | Where it lives                                                             | Meaning                                                       |
|--------------------------------|----------------------------------------------------------------------------|---------------------------------------------------------------|
| Mesh family                    | <code>meshes/*/definition.py</code> +<br>canonical <code>.msh</code> files | Geometry, physical tags, default material metadata            |
| <code>problem.mesh_path</code> | <code>case.toml</code>                                                     | Which family member or external mesh file to load             |
| <code>problem.elem_type</code> | <code>case.toml</code>                                                     | Which FE degree the runtime should use on that mesh           |
| Loader/elevation path          | <code>io.py</code> + <code>mesh/loader.py</code>                           | How the runtime lifts canonical tet4 storage to P1, P2, or P4 |

This is the core 3D mesh design change: **geometry/material family selection** and **finite-element degree selection** are no longer the same decision.

### Boundary labels

Mesh-family metadata declares which physical labels correspond to constrained  $x$ ,  $y$ , and  $z$  directions.

### Material identifiers

Element-wise material ids stay attached to the mesh and can be expanded into integration-point rows later.

### Node reorder

The same mesh can then be reordered for ownership without losing its physical meaning.

The rewrite does not hide physics labels behind generic partitioning. It keeps labels explicit, then reorders the discrete system around them.

# Speed Comparison



- The committed PETSc-vs-MATLAB performance report uses indirect SSR only and P2 elements only.
- PETSc runs use `mpirun -n 8` with `OMP_NUM_THREADS = 1`.
- MATLAB runs use `OMP_NUM_THREADS = 8` with the BoomerAMG MATLAB path on 8 threads.
- The three primary cases are homogeneous 3D SSR, heterogeneous 3D SSR, and heterogeneous 3D seepage SSR.
- The study is sequential and normalised into CSV before figures and tables are built.

The study answers “how does the committed P2 PETSc implementation compare under this controlled protocol?” It does **not** directly answer “how fast is the current default P4 architecture path?”.

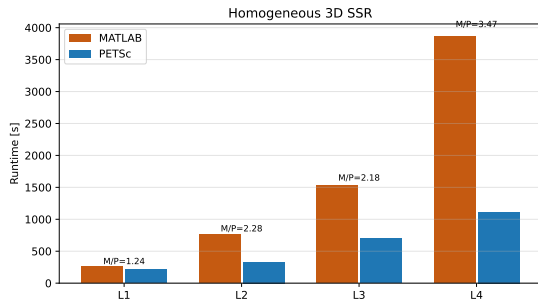
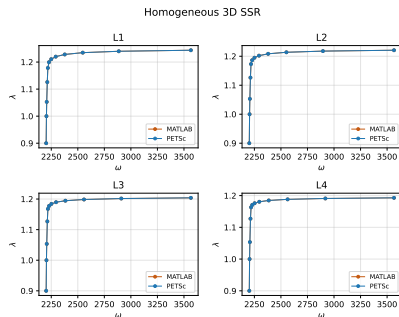
**Source:** docs/petsc\_matlab\_performance\_comparison: README and main LaTeX report

## Locked P2 Study: Headline View Across The Three Cases

| Case                         | Levels          | M/P gap        | Headline                                                                              |
|------------------------------|-----------------|----------------|---------------------------------------------------------------------------------------|
| Homogeneous 3D SSR           | L1 to L4        | 1.24× to 3.47× | PETSc pulls further ahead as the dry mesh ladder grows.                               |
| Heterogeneous 3D SSR         | L1 to L4        | 1.35× to 3.08× | Same trend on the dry case closest to the architecture anchor.                        |
| Heterogeneous 3D seepage SSR | concave_L2 only | 2.02×          | Positive completed level, but the next level remains part of the reported limitation. |

**Source:** committed P2 study generated tables in docs/petsc\_matlab\_performance\_comparison/generated

# Homogeneous 3D SSR: PETSc Pulls Ahead As The Mesh Grows

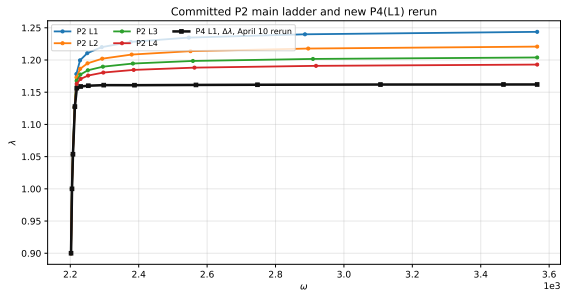


- PETSc runtime goes from 210.5 s at L1 to 1115.2 s at L4.
- MATLAB runtime goes from 261.8 s at L1 to 3865.1 s at L4.
- The MATLAB-over-PETSc ratio grows from  $1.24\times$  to  $3.47\times$ .

**Source:** presentation\_petsc\_rewrite/assets: homogeneous 3D continuation and timings from the committed study



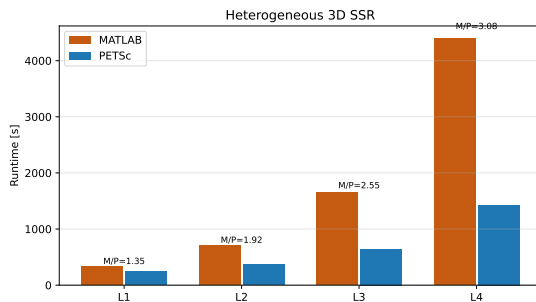
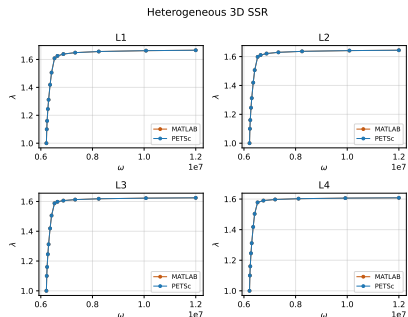
# Homogeneous 3D SSR: Committed P2 Ladder Versus The New P4(L1) Curve



- Every colored line is an available committed PETSc P2 study run for this case.
- The black line is the April 10 rerun on P4(L1) with the fixed PMG shell,  $\Delta\lambda$  stop, and the same  $\omega_{\max}$  as the committed study.
- This slide compares continuation shape under a higher-order discretisation, not runtime parity with the committed P2 protocol.

**Source:** committed docs/petsc\_matlab\_performance\_comparison/data/continuation\_curves.csv plus April 10, 2026 rerun artifacts in artifacts/comparisons/p4\_l1\_vs\_p2\_presentation\_20260410

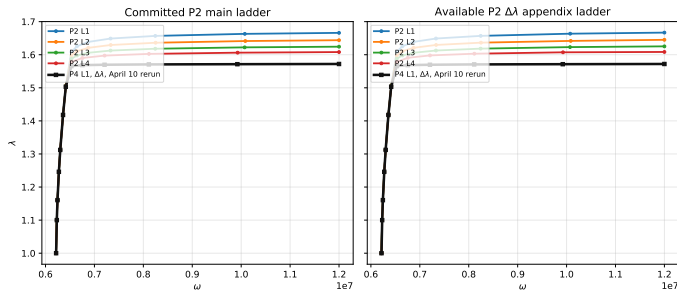
# Heterogeneous 3D SSR: Same Trend, Stronger Advantage On Larger Levels



- PETSc runtime grows from 247.3 s at L1 to 1430.2 s at L4.
- MATLAB runtime grows from 334.2 s at L1 to 4399.1 s at L4.
- The MATLAB-over-PETSc ratio grows from  $1.35\times$  to  $3.08\times$ .

**Source:** presentation\_petsc\_rewrite/assets: heterogeneous 3D continuation and timings from the committed study

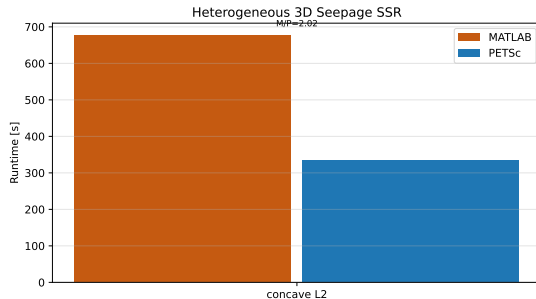
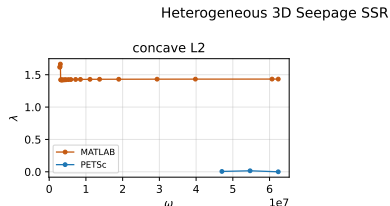
# Heterogeneous 3D SSR: Where The New P4(L1) Curve Sits Relative To P2



- Left: committed P2 main ladder. Right: available  $\Delta\lambda$  appendix ladder for the same dry heterogeneous case.
- The black line is the new P4(L1) rerun with the fixed PMG shell,  $\Delta\lambda = 10^{-4}$ , and the same  $\omega_{\max} = 1.2 \times 10^7$ .

**Source:** committed docs/petsc\_matlab\_performance\_comparison/data/continuation\_curves.csv plus April 10, 2026 rerun artifacts in artifacts/comparisons/p4\_l1\_vs\_p2\_presentation\_20260410

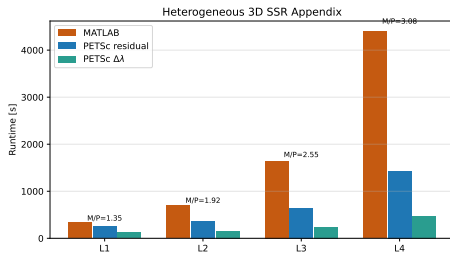
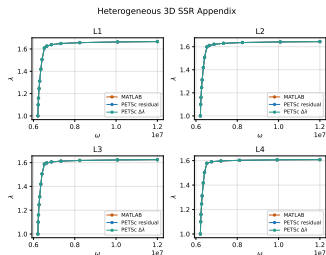
# Seepage 3D SSR: Only One Level Completed Under The Locked Protocol



- On concave\_L2, PETSc finished in 334.1 s and MATLAB in 676.3 s, a  $2.02\times$  advantage.
- Only this level completed under the committed mixed PMG-shell seepage study protocol.
- On this host, the local P4 rerun exceeded available memory during the SSR phase; the slide therefore keeps the committed P2 seepage evidence and marks P4 as skipped locally.

**Source:** presentation\_petsc\_rewrite/assets: seepage 3D continuation and timings from the committed study

# Additional Speedup Gain: The Delta-Lambda Stop Rule



- On the same heterogeneous dry study, the  $\Delta\lambda$  stop rule is  $2.0\times$  faster than the residual baseline at L1 and  $3.0\times$  faster at L4.
- Against MATLAB, that pushes the gap from  $2.7\times$  at L1 to  $9.3\times$  at L4.

This is a controlled “other speedup gain” slide. The study branch stays the same; only the stop policy changes.

**Source:** docs/petsc\_matlab\_performance\_comparison: delta-lambda figures and generated table

# Where To Edit The Code And Close

---

## Code Entry Points For Common Changes

| Question                                              | Primary entry point                                                                                                             |
|-------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Create or edit a benchmark                            | benchmarks/*/case.toml, then<br>src/slope_stability/core/run_config.py, then<br>src/slope_stability/cli/run_case_from_config.py |
| Where do mesh, BC labels, and default materials live? | meshes/*/definition.py plus<br>src/slope_stability/problem_assets.py                                                            |
| Change constitutive behavior                          | src/slope_stability/constitutive/problem.py and<br>src/slope_stability/constitutive/reduction.py                                |
| Change Newton or line search                          | src/slope_stability/nonlinear/newton.py and<br>src/slope_stability/nonlinear/damping.py                                         |
| Change continuation logic                             | src/slope_stability/continuation/indirect.py, direct.py,<br>limit_load.py                                                       |
| Change exports or notebooks                           | src/slope_stability/postprocess/* and<br>benchmarks/notebook_support.py                                                         |

## Algorithm Changes Live In A Few Concentrated Files

| Area                                    | Main edit surface                                                                                       |
|-----------------------------------------|---------------------------------------------------------------------------------------------------------|
| Constitutive law                        | constitutive/problem.py for stress/tangent logic and<br>constitutive/reduction.py for Davis reduction   |
| Newton solve                            | nonlinear/newton.py for SSR and LL Newton kernels;<br>nonlinear/damping.py for damping and Armijo logic |
| Continuation policy                     | continuation/indirect.py, continuation/direct.py, and<br>continuation/limit_load.py                     |
| Linear solver / PMG                     | linear/solver.py and linear/pmg.py                                                                      |
| Runner-side assembly and problem wiring | execution/asset_case/runner.py dispatching to generic mechanics<br>or seepage runners                   |
| Postprocess / visualisation             | postprocess/case_mesh.py, postprocess/field_exports.py, and<br>benchmarks/notebook_support.py           |



## What stayed familiar

SSR, Newton, constitutive reduction, and the benchmark families remain structurally familiar.

## What changed in the runtime

Configuration, ownership-aware assembly, solver separation, standard exports, and shared notebooks are now first-class interfaces.

## Shortest path to extension

Extension now starts from the benchmark folder, mesh-family metadata, concentrated algorithm files, and standard outputs instead of one large script.

# **Optional Paths, Caveats, And Additional Reading**

---

## Architecture Mainline Versus Performance Study Protocol

| Question             | Architecture mainline                 | Committed performance study                                        |
|----------------------|---------------------------------------|--------------------------------------------------------------------|
| Purpose              | Explain the current rewrite structure | Compare PETSc and MATLAB under one locked protocol                 |
| Case anchor          | default P4 benchmark                  | committed P2 report                                                |
| Element degree       | P4 default path                       | P2 only                                                            |
| Solver backend focus | pmg_shell, owned-row path             | PETSc MPI-8 versus MATLAB OMP-8                                    |
| What it proves       | How the rewrite is structured today   | How one committed PETSc branch compares numerically and in runtime |

## Optional And Inactive Paths Present In The Repository

- Direct SSR and LL continuation remain present and runnable for selected cases.
- BDDC, Hypr, GAMG, and multiple PMG variants all exist as selectable branches.
- Different constitutive ownership modes remain available beyond the overlap mainline.
- The map document keeps these paths in its annex so they do not distort the default-reading story.

| Path                          | Still present? | Why it is not the default story                                                                       |
|-------------------------------|----------------|-------------------------------------------------------------------------------------------------------|
| Global B' D B rebuild         | yes            | It matches the old MATLAB intuition better, but it is not the normal owned-row PMG route              |
| Legacy tangent kernel         | yes            | Kept for comparison and fallback, but the rows kernel is the production default for the mainline path |
| Legacy scatter-style assembly | yes            | Appendix material now that overlap-owned assembly is the default architectural choice                 |

| Branch        | Repository status  |     |      | Typical reason to look at it                                    |
|---------------|--------------------|-----|------|-----------------------------------------------------------------|
| hypre backend | active             |     |      | Classical AMG comparison against PMG-shell                      |
| gamg backend  | active             |     |      | PETSc-native algebraic multigrid studies                        |
| bddc backend  | active             |     |      | Domain decomposition experiments and MATIS-style subdomain work |
| DIRECT        | active<br>mainline | but | non- | Small-case validation, not distributed production solves        |

- The seepage section reports only `concave_L2`.
- The next seepage level reproduced a PETSc-side crash after hierarchy construction and outer-solver setup in the mixed shell hierarchy.
- Same-mesh fallback diagnostics avoided the crash but did not yield a usable converged benchmark within the locked study protocol.
- The report therefore keeps the completed comparison and states the limitation instead of smoothing it away.

| Level | PETSc residual [s] | PETSc $\Delta\lambda$ [s] | MATLAB [s] | Step counts  |
|-------|--------------------|---------------------------|------------|--------------|
| L1    | 247.304            | 122.460                   | 334.248    | 14 / 14 / 14 |
| L2    | 365.262            | 152.865                   | 700.772    | 14 / 14 / 14 |
| L3    | 645.949            | 239.822                   | 1645.273   | 14 / 14 / 14 |
| L4    | 1430.224           | 473.999                   | 4399.089   | 13 / 13 / 13 |

Read this appendix as a protocol sensitivity note, not as a replacement for the main report.



## Extra Source Map And Reading Order

1. docs/petsc\_matlab\_map/petsc\_to\_matlab\_solution\_map.tex
2. docs/config-scheme-3d.md
3. README.md
4. src/slope\_stability/cli/run\_case\_from\_config.py
5. src/slope\_stability/execution/asset\_case/runner.py
6. src/slope\_stability/fem/distributed\_tangent.py
7. src/slope\_stability/linear/solver.py
8. benchmarks/notebook\_support.py

For a fast orientation pass, the map document plus the default P4 architecture benchmark config are the highest-yield entry points.